

Two Proofs of Concept

Two independent, buildable experiments that de-risk the two-brain voice architecture from opposite ends. **LiveBridge** proves our stack can wear GPT-Live's clothes. **PromptSplit** proves we can get two-brain behavior from clients that only know how to send one prompt.

AT A GLANCE		
	POC 1 — LIVEBRIDGE	POC 2 — PROMPTSPLIT
Question it answers	Can an existing talker/thinker stack serve the GPT-Live API unchanged?	Can we derive two prompts from the one a Realtime client sends?
Protocol	GPT-Live	OpenAI Realtime
Client changes needed	None — stock Live clients	None — stock Realtime clients
Core deliverable	An adapter interface + conformance suite	A cached, hybrid prompt splitter + eval harness
Hardest problem	Full duplex behavior we may not have	Non-determinism and safety-rule loss
Rough size	6–9 engineer-weeks to P3	3–4 engineer-weeks to P2
Can run in parallel?	Yes — no shared code. They compose later (see §3)	

POC 1

LiveBridge

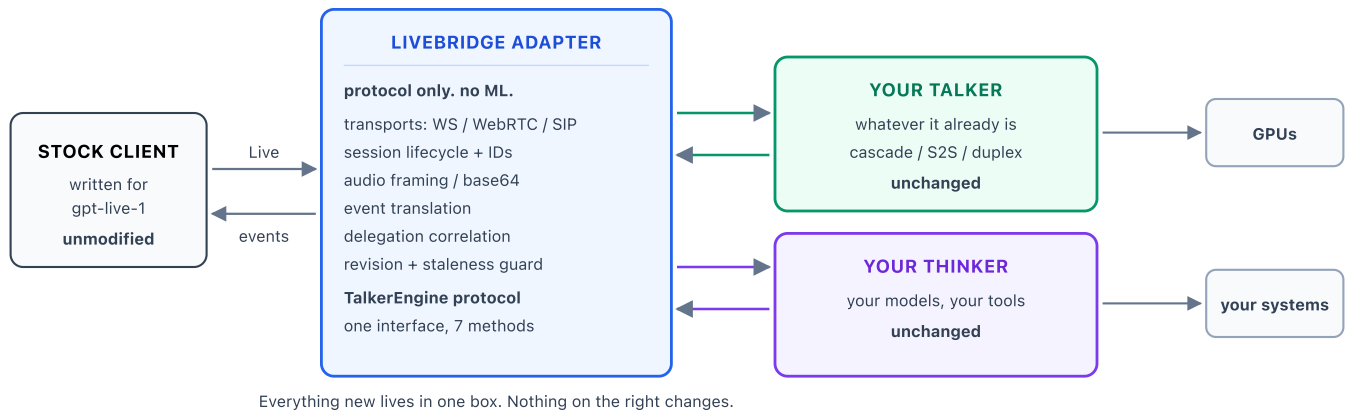
Wrap any existing talker/thinker architecture behind the GPT-Live API schema.

What it proves

That the GPT-Live schema is a **generic contract**, not an OpenAI-only interface — and that our own models can sit behind it. If this works, any client written against OpenAI's Live docs points at our host by changing one URL, and gets our models.

The strategic value is not the protocol work. It's that **we find out how far our Talker is from the behavior the contract implies**, early and cheaply, before that becomes a roadmap surprise.

Architecture



The core deliverable: one interface

The entire POC reduces to defining a narrow contract that any voice stack can implement. Get this right and the adapter is mechanical.

```
from typing import Protocol, AsyncIterator, Literal

class TalkerEngine(Protocol):
    """Anything that can hold a spoken conversation. Seven methods."""

    async def start(self, instructions: str, voice: str, fmt: AudioFormat) -> None:
        """Called once. instructions is IMMUTABLE for the session's life."""

    async def feed_audio(self, pcm: bytes) -> None:
        """Continuous. No commit, no turn signal -- we never get one."""

    def audio_out(self) -> AsyncIterator[bytes]:
        """Speech to play. We emit as session.output_audio.delta."""

    def transcripts(self) -> AsyncIterator[Transcript]:
        """Transcript(side='user'|'assistant', text, start_ms, end_ms)."""

    def delegations(self) -> AsyncIterator[DelegationRequest]:
        """The Talker saying 'I need help.' THE critical signal -- see below."""

    async def inject(self, kind: Literal["commentary", "thinking", "instructions"],
                    text: str) -> None:
```

```

        """Thinker result arrives as CONTEXT, not a return value.
        commentary -> may be spoken, paraphrased
        thinking    -> influences later speech, never spoken on arrival
        instructions -> changes behavior, may interrupt current speech"""

    async def close(self) -> None: ...

```

A reference implementation over our current cascade ships with the POC. A second implementation over a mock engine ships too, so the protocol can be tested with zero GPUs.

Event mapping

LIVE EVENT	DIR	ADAPTER DOES
session.start	in	Validate config; call <code>engine.start()</code> ; allocate <code>live_...</code> ID
session.started	out	Emit with resolved config. Gate all audio on this
session.input_audio.append	in	base64 decode, verify even byte length, <code>engine.feed_audio()</code>
session.output_audio.delta	out	base64 encode from <code>audio_out()</code> . Emit no "done" event
session.input_transcript.delta session.output_transcript.delta	out	Two independent streams. They may overlap — do not serialize them
session.delegation.created	out	Mint an <code>item_...</code> ID, stamp <code>offset_ms</code> , record the conversation revision. Send metadata only — never the task text
session.commentary.append session.thinking.append session.instructions.append	in	Validate <code>delegation_id</code> (null allowed), enforce 500-token cap, <code>engine.inject()</code> , emit the <code>*.appended</code> ack
session.update	in	Only <code>delegation</code> is mutable. Reject a type change with <code>immutable_field_update</code>
session.close / closed	both	Drain, emit final <code>usage</code> , then close. Clients wait for this
info	out	Our capability announcement. See the duplex section

The three hard problems

1 Full duplex we may not have

The Live contract assumes the model listens and speaks simultaneously. That's why no turn events exist. If our Talker is half-duplex, a client will wait for natural interruption that never comes — and get no error. Silent wrongness is the worst failure mode.

Proposal: a duplex ladder, and be honest about which rung we're on.

- L0** **Half duplex.** Speak, then listen. Contract violated. Acceptable only for P0 mock testing
- L1** **Interruptible.** VAD detects user speech mid-utterance; Talker stops within ~200ms. **This captures most of the perceived benefit** and is reachable with a cascade
- L2** **Backchannel.** Short "mm-hm" from a separate cheap path while the user speaks. Feels duplex without being duplex
- L3** **True duplex.** One model, both directions. Out of scope for this POC

Announce the rung on the `info` server event at session start. `info` is already in the Live event set, so this is a legitimate use of the protocol rather than an extension. Clients that ignore it still work; clients that read it can adapt. **P1 targets L1.**

2 Nothing tells us a turn ended

No `commit`, no `response.create`, no `response.cancel`. The client streams audio forever and expects us to work out the rest. Every timing decision moves server-side.

Proposal: own an explicit turn controller inside the adapter — endpointing, barge-in detection, and the speak/stay-quiet decision. Make it a swappable component so we can tune it without touching protocol code. Log every decision with `offset_ms` so sessions are replayable.

3 The Talker must decide to delegate

`session.delegation.created` has to come from somewhere. For a cascaded Talker this is a routing decision made per user turn.

Proposal: give the small Talker model exactly one tool, `delegate(user_intent)`. Small models emit tool calls far more reliably than they follow prose about when to defer, and we get the intent restatement for free. Route by allowlist — the Talker answers a closed set (greetings, repeats, clarifications) and delegates everything else — rather than asking it to judge its own competence.

Phases

PHASE	SCOPE	EXIT CRITERIA
P0	Protocol skeleton + mock engine. WebSocket only. No ML.	Echo engine holds a session; stock client connects, sends audio, receives audio, closes cleanly with final usage. Zero GPUs used.
P1	Real Talker, client delegation mode. L1 duplex.	A real conversation with one delegation round trip. Barge-in stops speech within 200ms.
P2	Responses delegation mode: nested <code>response.event</code> envelopes, client-executed tools.	A client running OpenAI's documented tool loop gets a tool call and its result is used.
P3	WebRTC transport + sideband attach.	Browser client works. Our server attaches to a browser session and injects an instruction.
P4	Hardening: reconnects, forks, recording, load.	Soak test at target concurrency with no session leaks.

Success criteria

One test, and it is falsifiable:

Take OpenAI's published Live sample client. Change only the base URL. It holds a working conversation against our server, including one delegation round trip, and closes with a valid `session.closed`.

Secondary: a **conformance suite** we publish alongside — a script that exercises every client event, asserts every required server event, and checks the immutability rules. That artifact outlives the POC and becomes our regression gate.

Risks

RISK	SEVERITY	RESPONSE
Duplex gap is wider than expected	high	This is the POC's <i>purpose</i> . Discovering it in P1 is a success, not a failure. The <code>info</code> capability signal is the honest mitigation
Live schema changes under us	med	It is five days old and gated at "alpha access" per partner docs. Pin to a dated snapshot of the docs; expect churn
Responses-mode envelope work is large	med	P2 is separable. Ship P1 in client mode and decide later whether Responses mode earns its cost
Nobody else adopts Live	med	Reduces the reach argument, not the design argument. The adapter interface is reusable regardless

PromptSplit

Turn the one prompt a Realtime client sends into a Talker prompt and a Thinker prompt, automatically.

What it proves

That we can run a two-brain architecture behind the **Realtime** API — the one that is already a de facto standard — **with no convention for clients to learn**. They send an ordinary prompt and an ordinary tool list. We derive the split.

This is the cheaper, lower-risk POC, and it unlocks the larger installed base. If only one gets built, build this one.

Flow

1	Client sends <code>session.update</code> with one <code>instructions</code> and a <code>tools</code> array	CLIENT
2	Hash (<code>instructions + tool signatures</code>) . Cache hit → done, 0ms.	US
3	Miss: rule pass first. Marked sections or standard headings split deterministically	US
4	Only unstructured prose goes to a small LLM, with structured output	SPLITTER
5	Tool manifest generated deterministically from the tool list — never by the LLM	US
6	Validate invariants, cache, configure Talker + Thinker	US

The insight that makes this viable: cache hit rate should approach 100%. A deployed app sends the *same* prompt on every session. The splitter runs once per distinct app configuration, not once per call. Add a pre-warm endpoint so clients can register a prompt at deploy time and the very first real call is fast too.

Hybrid design — rules first, LLM last

A pure LLM splitter is the obvious design and the wrong one. Three deterministic passes run before the model gets involved:

PASS	HANDLES	WHY NOT THE LLM
A	Explicit markers (<code># Responder</code> / <code># Planner</code>)	The client told us. Obeying is free and exact

PASS	HANDLES	WHY NOT THE LLM
B	Standard section headings (<i>Personality, Tools, Reasoning, Escalation...</i>)	A client following OpenAI's own prompt skeleton splits correctly by accident
C	Tool manifest for the Talker	Tools are structured data. Templating name + first sentence is exact, free, and cannot hallucinate a capability we lack
D	Remaining unstructured prose	Genuinely needs judgment. Only this reaches the LLM

```

SPLITTER_SCHEMA = {
  "type": "object",
  "properties": {
    "talker": {
      "type": "object",
      "properties": {
        "persona": {"type": "string"}, # tone, pace, warmth
        "style": {"type": "string"}, # length, formality
        "answer_directly_when": {"type": "array", "items": {"type": "string"}},
        "delegate_when": {"type": "array", "items": {"type": "string"}}
      },
      "required": ["persona", "delegate_when"]
    },
    "thinker": {
      "type": "object",
      "properties": {
        "rules": {"type": "string"}, # business logic, constraints
        "tool_guidance": {"type": "string"} # when to use what
      },
      "required": ["rules"]
    },
    "shared": {"type": "array", "items": {"type": "string"}},
    "confidence": {"type": "number"},
    "unassigned": {"type": "array", "items": {"type": "string"}}
  },
  "required": ["talker", "thinker", "shared", "confidence"]
}

```

`unassigned` is deliberate: text the splitter could not confidently place. It goes to **both** models and gets logged for review. Better duplicated than dropped.

Safety invariants — enforced in code, not by the model

These are checked after the split and before anything is used. A violation means fall back, not ship.

INVARIANT	ENFORCEMENT
No rule is lost	Every sentence of the input appears in at least one output, or in <code>unassigned</code> . Checked by coverage diff, not by trusting the model
Safety rules go to both	Keyword and classifier pre-pass tags refusal, escalation, identity, and compliance lines. These bypass the LLM and are force-appended to both halves
Talker never gets tool schemas	Structural assertion. The Talker's prompt is rejected if it contains a JSON Schema fragment
Talker never names a tool it cannot reach	Cross-check the manifest against the live tool list at session start
Low confidence degrades safely	Below threshold, the Talker gets a generic prompt that delegates <i>everything</i> . Slower, never wrong
Client prompt is data, not instruction	The splitter sees it inside a delimited block; structured output means it can never become free-form system text

Cold start

On a cache miss, audio may arrive before the split finishes. Three options, in order of preference:

1. **Pre-warm.** Clients register prompts at deploy time via a side endpoint. Best case, and free.
2. **Bootstrap prompt.** The Talker starts with a generic "acknowledge and delegate everything" prompt and we hot-swap the real one when the split lands. Correct throughout, just chattier for a second.
3. **Block the first turn.** Hold `session.updated` until the split completes. Simplest, but adds visible latency to session setup.

Ship option 2 as the default; it degrades in the direction of over-delegation, which is slow but never wrong.

Eval plan

The POC is not credible without measurement. Build a corpus of 40–60 real prompts spanning support, booking, ops, and compliance domains, each hand-split once to serve as reference.

METRIC	TARGET	HOW
Cache hit rate	> 95% steady state	Production counter. The economics depend on this
Split latency, p99	< 800ms	Only matters on a miss

METRIC	TARGET	HOW
Safety-rule retention	100%, no exceptions	Automated: every tagged safety line present in both halves. A hard gate
Coverage	> 98%	Input sentences appearing in some output
Delegation precision / recall	recall > 0.95	Replay scripted turns; did the Talker delegate when it should have? Recall matters more — over-delegation is slow, under-delegation is wrong
Talker prompt size	< 15% of input	If the Talker's half is large, the split did not work
Human preference vs. hand split	>= parity	Blind A/B on recorded conversations

Phases

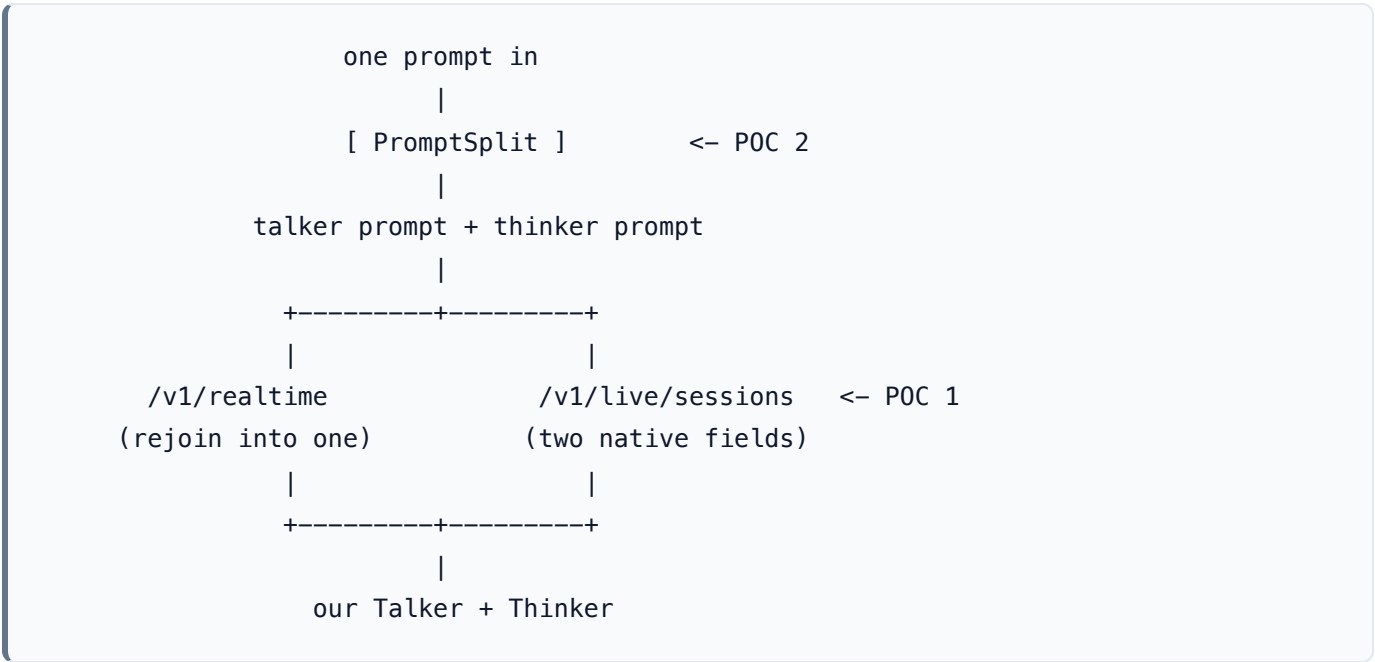
PHASE	SCOPE	EXIT CRITERIA
P0	Rule passes A–C only. No LLM.	Prompts with markers or standard headings split correctly. Establishes the baseline the LLM must beat
P1	Add LLM pass D + invariant checks + cache	All eval metrics collected on the corpus. Safety retention at 100%
P2	Wire into the Realtime server end to end	A stock Realtime client with one prompt gets two-brain behavior and never knows
P3	Pre-warm endpoint, observability, prompt-drift detection	Operable: we can see every split, diff it, and roll one back

Risks

RISK	SEVERITY	RESPONSE
A safety rule lands only in the Thinker	high	The reason safety lines never reach the LLM. Rule-tagged, force-duplicated, hard-gated in eval
Non-determinism across sessions	med	Content-hash caching makes a given prompt split identically forever. Version the cache key with the splitter prompt so upgrades are explicit
Client prompt injection via the splitter	med	Structured output only; delimited input block; no free-form passthrough
Splitter quality plateaus below hand-split	med	P0 baseline still ships value on its own. Marker support gives sophisticated clients an escape hatch
Prompt drift after a client edit	low	New hash, new split, automatic. Log the diff

3 How they fit together

They are independent to build and complementary to ship. PromptSplit produces exactly the pair of prompts LiveBridge consumes.



Run both and one server serves both protocols from one set of models. A Realtime client sends one prompt and we split it; a Live client sends two and we skip the splitter entirely.

Sequencing

IF OUR PRIORITY IS...	THEN
Reach	PromptSplit first. Realtime is the standard with real adoption — vLLM and Hugging Face already speak it. Lower cost, lower risk, larger audience
Learning about our own stack	LiveBridge P0–P1 first. It tells us the duplex gap, which is the single biggest unknown in the roadmap
Both, with two engineers	Parallel. No shared code, no shared risk. Converge at PromptSplit P2 / LiveBridge P1

One shared dependency, worth building once. Both POCs need conversation-revision tracking so a stale Thinker result is discarded when the user changes their mind mid-request. Build it as a standalone module before either POC starts. It is small, it is the most common bug in this architecture, and OpenAI's own reference code guards for it.

Companion to the **Two-Brain Voice Assistant** design note.

Protocol details: [Live delegation](#) · [Live migration](#) · [Live prompting](#) · [Realtime conversations](#)